

Hệ thống hỏi đáp về Đại học Quốc gia Hà Nội dựa trên Retrieval-Augmented Generation

Nguyễn Đăng Dương Bùi Hải Đăng Tôn Thành Đạt
MSSV: 23020350 MSSV: 23020356 MSSV: 23020354
23020350@vnu.edu.vn 23020356@vnu.edu.vn 23020354@vnu.edu.vn

Abstract

Báo cáo này mô tả, một hệ thống Retrieval - Augmented Generation (RAG) trả lời câu hỏi về Đại học Quốc gia Hà Nội (Vietnam National University). Hệ thống thu thập dữ liệu từ năm trang thông tin chính thức của VNU, làm sạch dữ liệu theo từng loại tài liệu, chia văn bản thành các chunk theo cấu trúc ngữ nghĩa, lập chỉ mục vector bằng FAISS, truy xuất top-10 chunk bằng mô hình BAAI/bge-m3, xếp hạng lại top-3 chunk bằng cross-encoder, sau đó sinh câu trả lời ngắn gọn bằng mô hình ngôn ngữ chỉ dựa trên ngữ cảnh truy xuất. Tập dữ liệu của nhóm gồm 41 chunk, 28 cặp hỏi-đáp huấn luyện few-shot, 12 cặp hỏi-đáp kiểm thử và một tập con 8 câu cho đánh giá inter-annotator agreement (IAA). Kết quả IAA đạt canonical exact agreement 0.7500 và canonical token F1 0.8047, cho thấy chất lượng chú thích tương đối ổn. Trong khi đó, kết quả trên tập test đạt Exact Match 0.4167, F1 Score 0.6673 và Recall 0.7167, cho thấy dù câu trả lời không khớp nguyên văn, hệ thống vẫn lấy được khá nhiều thông tin đúng và có mức độ trùng khớp nội dung tương đối tốt với đáp án tham chiếu.

1 Giới thiệu

Các hệ thống hỏi đáp trong miền lĩnh vực hẹp thường cần câu trả lời vừa chính xác theo tài liệu, vừa có khả năng diễn đạt tự nhiên. Nếu chỉ dùng mô hình ngôn ngữ lớn, hệ thống có thể sinh câu trả lời không có trong nguồn dữ liệu. Ngược lại, nếu chỉ dùng tìm kiếm các từ khóa, hệ thống khó xử lý các câu hỏi diễn đạt khác với văn bản gốc. Retrieval-Augmented Generation (RAG) giải quyết vấn đề này bằng cách kết hợp truy xuất tài liệu và sinh câu trả lời

có điều kiện trên ngữ cảnh [Lewis et al., 2020].

Trong bài này, nhóm xây dựng hệ thống RAG cho miền tri thức về ĐHQGHN. Đầu vào là câu hỏi tiếng Việt của người dùng. Đầu ra là câu trả lời ngắn, trực tiếp, và chỉ sử dụng thông tin trong các đoạn văn bản được truy xuất. Hệ thống được thiết kế theo pipeline end-to-end gồm: thu thập dữ liệu, làm sạch, chunking, embedding, FAISS retrieval, cross-encoder reranking, few-shot prompting và generation.

Đóng góp chính của hệ thống gồm:

1. Xây dựng bộ dữ liệu về ĐHQGHN từ 5 trang chính thức, bao phủ lịch sử, sứ mệnh-tầm nhìn, chiến lược phát triển, thi đua khen thưởng và thống kê nhân sự.
2. Thiết kế chiến lược chunking theo cấu trúc nội dung thay vì cắt cố định: lịch sử theo mốc năm, chiến lược theo mục/trụ cột, thống kê theo nhóm đơn vị.
3. Sử dụng few-shot learning kết hợp dense retrieval bằng BAAI/bge-m3, FAISS [Johnson et al., 2019], reranking bằng cross-encoder [Nogueira and Cho, 2019], và mô hình sinh câu trả lời có ràng buộc theo ngữ cảnh.
4. Tạo tập QA train/test và đánh giá chất lượng chú thích bằng IAA cho câu trả lời dạng tự do.

2 Dữ liệu

2.1 Nguồn dữ liệu

Tập dữ liệu của hệ thống được thu thập trực tiếp từ năm trang chuyên mục giới thiệu và tổng quan trên cổng thông tin điện

tử chính thức của Đại học Quốc gia Hà Nội (VNU). **Bảng 1** trình bày chi tiết về quy mô dữ liệu sau khi được làm sạch và số lượng phân đoạn (chunk) tương ứng được sinh ra từ mỗi nguồn.

2.2 Tiền xử lý

Quá trình làm sạch dữ liệu (trong `src/data_processing/data_cleaner.py`) đóng vai trò quan trọng trong việc nâng cao chất lượng đầu vào. Đầu tiên, toàn bộ văn bản trải qua bước tiền xử lý chung nhằm chuẩn hóa ký tự xuống dòng, loại bỏ khoảng trắng dư thừa và gộp các dòng trống liên tiếp. Sau đó, tùy thuộc vào đặc thù của từng loại tài liệu, các quy tắc xử lý riêng biệt được áp dụng:

- **Lịch sử:** Loại bỏ các dòng chú thích ảnh (caption), thông tin nhiễu, đồng thời ghép nối các câu bị ngắt dòng sai quy tắc nhằm đảm bảo tính mạch lạc, nhưng vẫn duy trì nguyên vẹn các mốc thời gian.
- **Chiến lược phát triển:** Lọc bỏ các liên kết điều hướng (như “Tải bản đầy đủ”) và nhãn infographic. Đặc biệt, dữ liệu dạng bảng biểu thể hiện các chỉ tiêu chiến lược được chuyển đổi thành dạng câu văn có cấu trúc tự nhiên để mô hình dễ trích xuất.
- **Thống kê nhân sự:** Tương tự, dữ liệu từ các bảng thống kê được tự động diễn đạt lại thành ngôn ngữ tự nhiên và nhóm theo từng đơn vị cụ thể.
- **Sứ mệnh – Tầm nhìn và Thi đua khen thưởng:** Do bản chất thông tin ngắn gọn và đã tương đối chuẩn xác, hai nhóm dữ liệu này chỉ cần qua bước chuẩn hóa định dạng cơ bản.

2.3 Chunking theo ngữ nghĩa

Để khắc phục hạn chế của việc cắt văn bản theo độ dài cố định làm đứt gãy mạch ý, hệ thống áp dụng kỹ thuật phân mảnh dựa trên cấu trúc nội dung ngữ nghĩa (`src/data_processing/chunker.py`). Phương pháp này đảm bảo mỗi chunk duy trì được một ngữ cảnh trọn vẹn, từ đó cải thiện đáng kể độ chính xác ở bước truy xuất (retrieval). Cụ thể, đối với tài

liệu lịch sử, văn bản được phân tách dựa trên các dấu mốc thời gian (ví dụ: Năm XXXX hoặc Giai đoạn XXXX). Với văn bản chiến lược, dữ liệu được chia theo các đề mục lớn (đánh số La Mã); trường hợp một đoạn có độ dài vượt quá 1.500 ký tự, đoạn đó sẽ tiếp tục được chia nhỏ đến cấp tiểu mục. Đối với số liệu nhân sự, các dòng văn bản được gom nhóm hợp lý theo loại hình đơn vị (trường đại học, viện nghiên cứu, trung tâm, ban quản lý...). Kết quả phân mảnh được lưu trữ dưới định dạng JSONL, trong đó mỗi đoạn đều được gắn kèm các siêu dữ liệu (metadata) hỗ trợ truy xuất như `source`, `topic` và `section`. Về mặt thống kê, độ dài trung bình của mỗi chunk là xấp xỉ 358 token; chunk ngắn nhất chứa 37 token và chunk dài nhất chứa 1.729 token. Phân phối kích thước này đặc biệt phù hợp với bài toán hỏi đáp đóng miền (closed-domain QA), do nó bảo toàn được vùng ngữ cảnh đủ rộng để hệ thống có thể trả lời các câu hỏi phức tạp về mốc sự kiện, số liệu chi tiết hay danh sách các tổ chức trực thuộc.

3 Phương pháp

3.1 Triển khai và các nguồn tài nguyên bên ngoài

Hệ thống của chúng tôi được xây dựng theo khung Retrieval-Augmented Generation do Lewis et al. [2020] đề xuất. Cơ sở tri thức được thu thập từ các trang web chính thức của ĐHQGHN, bao gồm các trang về lịch sử, sứ mệnh và tầm nhìn, chiến lược phát triển, thi đua khen thưởng và số liệu thống kê của nhà trường [Vietnam National University, Hanoi, 2026c,d,b,a,e]. Trong quá trình thu thập dữ liệu, chúng tôi sử dụng `requests` để tải các trang HTML và `BeautifulSoup` để phân tích cú pháp cũng như trích xuất nội dung văn bản [Python Requests, 2026, Richardson, 2026].

Đối với bước truy xuất, các tài liệu và câu hỏi được mã hóa bằng BGE-M3, một mô hình embedding đa ngôn ngữ được thiết kế cho truy xuất dense, truy xuất sparse và truy xuất multi-vector [Chen et al., 2024, BAAI, 2024]. Chỉ mục vector được xây dựng bằng FAISS nhằm thực hiện tìm kiếm

File nguồn	Chủ đề	Số token sau làm sạch	Số chunk
lich-su.txt	Lịch sử ĐHQGHN	9,822	23
chien-luoc-phat-trien.txt	Chiến lược phát triển ĐHQGHN	3,424	11
so-lieu-thong-ke.txt	Số liệu thống kê nhân sự ĐHQGHN	1,098	5
su-mang-tam-nhin.txt	Sứ mệnh, tầm nhìn, giá trị cốt lõi	121	1
thi-dua-khen-thuong.txt	Thành tích thi đua khen thưởng	189	1
Tổng	5 chủ đề	14,654	41

Table 1: Thống kê dữ liệu sau bước làm sạch và chunking.

tương đồng một cách hiệu quả [Johnson et al., 2019]. Chúng tôi sử dụng thêm một bước xếp hạng lại đoạn văn dựa trên hướng tiếp cận BERT re-ranking [Nogueira and Cho, 2019]. Câu trả lời cuối cùng được sinh bằng Qwen2.5-Instruct, một mô hình ngôn ngữ lớn mã nguồn mở có sẵn trên HuggingFace [Yang et al., 2024, Qwen Team, 2024]. Quá trình triển khai cũng sử dụng các thư viện HuggingFace Transformers và Sentence-Transformers [Wolf et al., 2020, Reimers and Gurevych, 2019]. Việc đánh giá được thực hiện bằng các độ đo Exact Match, F1 và Recall ở mức token, dựa trên giao thức đánh giá của SQuAD [Rajpurkar et al., 2016].

3.2 Tổng quan pipeline

Pipeline của hệ thống tuân theo các bước như sau:

Raw Text → Data Cleaning → Smart Chunking → Embedding → FAISS Index → Retrieval top-10 → Reranking top-3 → Generating answers.

Bảng 2 bên dưới trình bày chi tiết các thành phần chính của hệ thống.

3.3 Dense retrieval

Gọi q là câu hỏi và d_i là chunk thứ i . Bộ mã hóa sinh embedding chuẩn hóa e_q và e_i . Vì vector đã được chuẩn hóa, hệ thống dùng inner product trong FAISS như cosine similarity:

$$s(q, d_i) = e_q^\top e_i. \quad (1)$$

Với mỗi câu hỏi, FAISS trả về top-10 chunk có điểm $s(q, d_i)$ cao nhất. Hệ thống

dùng IndexFlatIP, tức là tìm kiếm chính xác trên toàn bộ vector thay vì approximate nearest neighbor. Cách này phù hợp với tập 41 chunk vì kích thước dữ liệu nhỏ.

3.4 Reranking

Dense retrieval có ưu điểm nhanh nhưng đôi khi chỉ dựa trên tương đồng biểu diễn toàn cục. Vì vậy, hệ thống dùng cross-encoder để chấm điểm lại từng cặp (q, d_i) . Cross-encoder nhận cả câu hỏi và chunk trong cùng một đầu vào, nên có thể mô hình hóa tương tác token-token tốt hơn bi-encoder. Sau reranking, chỉ top-3 chunk được đưa vào prompt sinh câu trả lời.

3.5 Generating answers

Prompt generation có ba ràng buộc quan trọng. Thứ nhất, mô hình đóng vai trò trợ lý chuyên gia trả lời câu hỏi về ĐHQGHN. Thứ hai, mô hình chỉ được dùng thông tin trong các ngữ cảnh được cung cấp. Thứ ba, nếu ngữ cảnh không chứa câu trả lời, mô hình phải trả lời “Không tìm thấy thông tin trong tài liệu.” Prompt cũng chèn các ví dụ few-shot từ tập train để định hướng phong cách trả lời ngắn gọn.

4 Tập hỏi–đáp và chú thích

4.1 Tạo tập QA

Các cặp hỏi–đáp được biên soạn thủ công dựa trên văn bản đã qua làm sạch (src/data_annotation/generate_qa.py), bao quát năm chủ đề chính: lịch sử, sứ mệnh–tầm nhìn, chiến lược phát triển, thi đua khen thưởng và số liệu thống kê. Tổng cộng có 40 câu hỏi được chia ngẫu nhiên

Thành phần	Cài đặt trong hệ thống
Thu thập dữ liệu	Sử dụng requests và BeautifulSoup trích xuất nội dung, sau đó lấy thông tin trong phần post-content, main-content hoặc body của website.
Làm sạch dữ liệu	Các hàm làm sạch riêng cho từng nguồn dữ liệu: lịch sử, sứ mệnh, chiến lược phát triển, thi đua khen thưởng, và số liệu thống kê trong data_cleaner.py.
Chunking	Chunk theo mốc năm, section, trụ cột chiến lược và nhóm đơn vị; lưu thành file chunks.jsonl.
Embedding	BAAI/bge-m3, mã hóa chunk và query thành vector dense.
Vector database	FAISS IndexFlatIP; vector được chuẩn hóa nên inner product tương đương như cosine similarity.
Retrieval	Lấy top-10 chunk ứng viên theo điểm tương đồng trên dense vectors.
Reranking	Sử dụng Cross-encoder BAAI/bge-reranker-base, chọn top-3 chunk có điểm reranking cao nhất.
Generation	Mô hình được dùng trong hệ thống là Qwen/Qwen2.5-7B-Instruct; prompt yêu cầu trả lời ngắn gọn và chỉ dựa trên ngữ cảnh được cung cấp để tìm câu trả lời.
Few-shot	Lấy tối đa 5 ví dụ đầu tiên từ data/train/questions.txt và data/train/reference_answers.txt.

Table 2: Các thành phần chính trong hệ thống RAG.

Split	Số câu hỏi	Số câu trả lời tham chiếu	Chỉ số	Giá trị
Train	28	28	Exact agreement	0.2500
Test	12	12	Canonical exact agreement	0.7500
IAA subset	8	8	Token F1	0.6410
			Canonical token F1	0.8047
			Số disagreement	2 / 8

Table 3: Thống kê tập hỏi-đáp dùng cho few-shot, đánh giá hệ thống và đánh giá IAA.

Table 4: Kết quả inter-annotator agreement trên tập con 8 câu.

thành tập train (28 câu) và tập test (12 câu), chi tiết tại **Bảng 3**.

Trong hệ thống này, tập train không dùng để tinh chỉnh (fine-tune) mô hình mà đóng vai trò cung cấp các ví dụ minh họa (in-context learning) trực tiếp trong prompt. Tập test được sử dụng hoàn toàn để sinh kết quả dự đoán và đánh giá đối chiếu với câu trả lời tham chiếu.

4.2 Inter-annotator agreement

Vì các câu trả lời tham chiếu được viết ở dạng tự do (free-form), việc đo lường độ đồng thuận giữa những người dán nhãn (IAA) không thể chỉ dựa vào phép khớp chuỗi tuyệt đối. Do đó, kết quả được đánh giá qua hai mức chuẩn hóa:

- **Normal exact/F1:** Chuyển sang chữ thường, xóa dấu câu, lược bỏ nội dung trong ngoặc và chuẩn hóa các khoảng trắng thừa.
- **Canonical exact/F1:** Kế thừa bước trên, đồng thời lược bỏ các tiền tố phụ (như “năm”, “ngày”, “có”), chuẩn hóa định dạng số và loại bỏ các ngữ cảnh thừa phía sau cụm danh từ.

Bảng 4 trình bày kết quả IAA trên 8 câu hỏi được hai annotator gán nhãn độc lập.

Kết quả cho thấy tỷ lệ khớp tuyệt đối (exact agreement) ban đầu khá thấp do sự khác biệt tự nhiên trong văn phong diễn đạt (ví dụ: “1906” so với “Năm 1906”). Tuy nhiên, sau khi áp dụng chuẩn hóa sâu, tỷ lệ đồng thuận đã tăng mạnh lên 0.7500 và chỉ số Token F1 đạt 0.8047. Qua phân tích, hai trường hợp bất đồng chính đều xuất phát từ sai sót thông tin của người gán nhãn: một người nhầm năm thành lập Đại học Đông Dương (1996 thay vì 1906), và trường hợp còn lại liệt kê thiếu tên các trường đại học thành viên khi sắp xếp lại ĐHQGHN.

5 Đánh giá

5.1 Metric

Hệ thống dùng ba metric tự động, tương tự cách đánh giá extractive QA, mục 6.1 của SQuAD [Rajpurkar et al., 2016]:

- **Exact Match (EM):** bằng 1 nếu câu trả lời dự đoán khớp hoàn toàn với câu trả

Metric	Giá trị
Exact Match	0.4167
F1 Score	0.6673
Recall	0.7167

Table 5: Bảng kết quả end-to-end cần được cập nhật sau khi sinh file prediction.

lời tham chiếu sau chuẩn hóa, ngược lại bằng 0.

- **Token-level F1:** trung bình điều hòa giữa precision và recall ở mức token.
- **Recall:** tỷ lệ token trong câu trả lời tham chiếu xuất hiện trong câu trả lời dự đoán.

Với các câu trả lời tham chiếu có nhiều biến thể phân tách bằng dấu chấm phẩy, hệ thống lấy điểm cao nhất giữa prediction và từng reference candidate.

5.2 Quy trình chạy evaluation

Quy trình đánh giá end-to-end được cài đặt để đọc câu hỏi test, sau đó gọi đến RAGSystem cho từng câu, kết quả trả về được lưu vào system_outputs/system_output_1.txt, cuối cùng gọi evaluate.py để tính giá trị cho Exact match, F1 và Recall.

Bảng 5 thể hiện kết quả đạt được trên tập test với 3 metrics EM, F1 và Recall.

6 Phân tích lỗi

Từ cấu trúc của hệ thống, có ba nhóm lỗi có khả năng xảy ra.

Lỗi truy xuất. Nếu chunk đúng không nằm trong top-10 kết quả đầu tiên, reranker và generator sẽ không có đủ thông tin để trả lời. Điều này có thể xảy ra với câu hỏi hỏi về số liệu cụ thể, vì các con số trong bảng thống kê dễ bị mất ngữ cảnh nếu chunking không gom đủ dòng liên quan.

Lỗi reranking. Cross-encoder giúp lọc kết quả tốt hơn, nhưng nếu câu hỏi có nhiều thực thể gần giống nhau, reranker có thể ưu tiên đoạn văn có ngôn ngữ tương tự câu hỏi hơn là đoạn chứa đáp án chính xác.

Lỗi generation. Mặc dù prompt yêu cầu chỉ dùng ngữ cảnh, mô hình sinh vẫn có thể diễn giải dài hơn reference hoặc thêm từ nối, làm giảm EM dù nội dung đúng. Đây là lý do F1 và recall cần được sử dụng cùng với EM.

7 Hướng cải thiện trong tương lai

Hệ thống hiện còn một số hạn chế:

- **Mở rộng trên kho dữ liệu lớn:** Tập dữ liệu hiện dùng còn nhỏ, gồm 5 nguồn tài liệu và 41 chunk, nên chưa đánh giá được khả năng mở rộng.
- **Mở rộng số lượng mẫu trên tập test:** Tập test hiện đang có 12 câu hỏi, chưa đủ đa dạng để kết luận chắc chắn về chất lượng tổng quát của hệ thống.
- **Xử lý sạch hơn ở phần chunking:** Chunk dài nhất có 1,729 token, có thể làm prompt dài và chứa thông tin dư thừa.
- **Trích dẫn nguồn cụ thể làm minh chứng:** Hệ thống chưa trả về trích dẫn nguồn cụ thể trong câu trả lời cuối, trong khi đây là tính năng quan trọng đối với QA dựa trên tài liệu.

8 Kết luận

Báo cáo đã trình bày một pipeline RAG end-to-end cho hỏi đáp về ĐHQGHN. Điểm mạnh của hệ thống nằm ở bước xử lý dữ liệu miền hẹp: làm sạch theo từng loại tài liệu, chunking theo cấu trúc ngữ nghĩa, kết hợp dense retrieval và reranking trước khi sinh câu trả lời. Tập QA train/test và đánh giá IAA đã được xây dựng, trong đó IAA sau chuẩn hóa đạt canonical token F1 0.8047. Kết quả trên tập test cho thấy phương pháp RAG có lợi thế trong các câu hỏi cần kiến thức cụ thể từ tài liệu, vì hệ thống có thể truy xuất thông tin liên quan trước khi sinh câu trả lời. Kết hợp với few-shot learning, hệ thống đạt khả năng bao phủ thông tin và giảm nguy cơ trả lời thiếu căn cứ. Điều này được thể hiện qua Recall đạt 0.7167, cho thấy hệ thống truy xuất và đưa vào câu trả lời phần lớn các nội dung cần thiết. Tuy nhiên, Exact Match chỉ đạt 0.4167 cho thấy RAG vẫn còn hạn chế ở khả

năng tạo câu trả lời chính xác hoàn toàn theo đáp án tham chiếu, đặc biệt khi câu trả lời yêu cầu đúng chi tiết hoặc đúng cách diễn đạt.

Acknowledgments

Dữ liệu được sử dụng trong bài lấy từ các trang thông tin công khai của Đại học Quốc gia Hà Nội.

References

BAAI. 2024. BGE-M3. <https://huggingface.co/BAAI/bge-m3>. Accessed: 2026-05-23.

Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*.

Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*.

Python Requests. 2026. Requests: Http for humans. <https://requests.readthedocs.io/>. Accessed: 2026-05-23.

Qwen Team. 2024. Qwen2.5-7B-Instruct. <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>. Accessed: 2026-05-23.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392.

Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.

Leonard Richardson. 2026. Beautiful soup documentation. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>. Accessed: 2026-05-23.

Vietnam National University, Hanoi. 2026a. Awards and emulation of vietnam national university, hanoi. <https://vnu.edu.vn/gioi-thieu/tong-quan/thi-dua-khen-thuong>. Accessed: 2026-05-23.

Vietnam National University, Hanoi. 2026b. Development strategy of vietnam national university, hanoi. <https://vnu.edu.vn/gioi-thieu/tong-quan/chien-luoc-phat-trien>. Accessed: 2026-05-23.

Vietnam National University, Hanoi. 2026c. History of vietnam national university, hanoi. <https://vnu.edu.vn/gioi-thieu/tong-quan/lich-su>. Accessed: 2026-05-23.

Vietnam National University, Hanoi. 2026d. Mission and vision of vietnam national university, hanoi. <https://vnu.edu.vn/gioi-thieu/tong-quan/su-mang-tam-nhin>. Accessed: 2026-05-23.

Vietnam National University, Hanoi. 2026e. Statistics of vietnam national university, hanoi. <https://vnu.edu.vn/gioi-thieu/tong-quan/so-lieu-thong-ke>. Accessed: 2026-05-23.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, and 24 others. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

A Lệnh chạy hệ thống

```
pip install requests beautifulsoup4
↪ sentence-transformers faiss-cpu

python src/data_collection/crawler.py
```

```
python
↳ src/data_processing/data_cleaner.py
python src/data_processing/chunker.py
python
↳ src/data_annotation/generate_qa.py
python src/model/rag_pipeline.py
python src/evaluation/run_eval.py
python src/evaluation/compute_iaa.py
```